Universidade de Brasília

Departamento de Ciência da Computação

Relatório do Trabalho 1 de Programação Concorrente

Evelyn Soares Pereira 17/0102785

Brasília
9 de novembro de 2026

1 Introdução

Em programação concorrente, *threads* que cooperam por memória compartilhada correm o risco de *condições de corrida*: se duas delas acessam o mesmo dado sem coordenação e pelo menos uma escreve, o resultado passa a depender da ordem em que o escalonador intercala suas instruções. Como essa ordem muda a cada execução, os erros aparecem em uma execução e podem sumir na seguinte. Para evitá-los, as *threads* são sincronizadas com mecanismos como *locks*, variáveis de condição e semáforos.

Este trabalho aplica esses mecanismos a um problema de Manna e Pnueli, apresentado por Ben-Ari (2006, Apêndice C, exercício 8): calcular o coeficiente binomial $\binom{n}{k}$ com dois processos, um para o numerador e outro para o denominador. O denominador entrega resultados parciais ao numerador, que divide assim que os recebe, impedindo que os valores intermediários cresçam demais. Essa comunicação acontece por memória compartilhada e, sem sincronização, está sujeita a condições de corrida. A solução foi implementada em C com POSIX Pthreads. A Seção 2 formaliza o problema, a Seção 3 descreve o algoritmo, a implementação e os testes, e a Seção 4 conclui o trabalho.

2 Formalização do Problema Proposto

2.1 O Coeficiente Binomial

Para inteiros $0 \le k \le n$,

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!} = \frac{n \cdot (n-1) \cdots (n-k+1)}{1 \cdot 2 \cdots k}, \tag{1}$$

ou seja, há k fatores no numerador e k no denominador. Calcular o numerador inteiro e só dividir no final é inviável mesmo para valores modestos: em $\binom{30}{15}$, o numerador $30 \cdot 29 \cdots 16 \approx 2{,}0 \times 10^{20}$ não cabe em 64 bits ($\approx 1{,}8 \times 10^{19}$), embora o resultado ($155\,117\,520$) caiba em 32. A proposta do exercício é intercalar as operações: no passo i, multiplicar o valor acumulado pelo i-ésimo fator do numerador e dividir pelo i-ésimo fator do denominador (Tabela 1).

Tabela 1: Cálculo de $\binom{10}{3}$ intercalando multiplicações e divisões.

Passo i	Multiplica por	Antes da divisão	Divide por	Depois da divisão
1	10	10	1	10
2	9	90	2	45
3	8	360	3	120

Como a divisão inteira em C descarta o resto sem aviso, esse método só funciona se toda divisão intermediária for exata, o que é provado a seguir.

2.2 Divisibilidade dos Resultados Parciais

Chamamos de $P(i, j)$ o produto de i inteiros consecutivos começando em j:

$$P(i, j) = j \cdot (j+1) \cdots (j+i-1). \qquad \text{Por exemplo, } P(3, 5) = 5 \cdot 6 \cdot 7 = 210.$$

Lema 1. *Para todos os inteiros $i \ge 1$ e $j \ge 1$, $i!$ divide $P(i, j)$.*

A demonstração usa uma identidade auxiliar. Para $i \geq 2$, os produtos $P(i, j)$ e $P(i, j+1)$ têm em comum os fatores $(j+1) \cdots (j+i-1)$, que formam $P(i-1,\, j+1)$. Assim,

$$P(i, j) = j \cdot P(i-1,\, j+1) \qquad \text{e} \qquad P(i, j+1) = P(i-1,\, j+1) \cdot (j+i),$$

e, subtraindo uma da outra,

$$P(i, j+1) - P(i, j) = \big[(j+i) - j\big] \cdot P(i-1,\, j+1) = i \cdot P(i-1,\, j+1). \tag{2}$$

Por exemplo, $P(3, 6) - P(3, 5) = 6 \cdot 7 \cdot 8 - 5 \cdot 6 \cdot 7 = (8-5) \cdot 6 \cdot 7 = 3 \cdot P(2, 6)$.

Demonstração. Por indução em i.

Base ($i = 1$): $P(1, j) = j$, e $1! = 1$ divide qualquer inteiro.

Passo: seja $i \geq 2$ e suponha, como hipótese de indução, que $(i-1)!$ divide $P(i-1, m)$ para todo $m \geq 1$. Vamos mostrar, agora por indução em j, que $i!$ divide $P(i, j)$ para todo $j \geq 1$.

- $j = 1$: $P(i, 1) = 1 \cdot 2 \cdots i = i!$, que é divisível por $i!$.
- De j para $j+1$: suponha que $i!$ divide $P(i, j)$. Pela identidade (2),

  $$P(i, j+1) = P(i, j) + i \cdot P(i-1,\, j+1).$$

  A primeira parcela é múltipla de $i!$ pela suposição. Na segunda, a hipótese de indução (com $m = j+1$) dá $P(i-1,\, j+1) = c \cdot (i-1)!$ para algum inteiro c, logo $i \cdot P(i-1,\, j+1) = c \cdot i \cdot (i-1)! = c \cdot i!$. A soma de dois múltiplos de $i!$ é múltipla de $i!$. □

Aplicação ao problema. Depois de i multiplicações, o numerador acumulou $N_i = n \cdot (n-1) \cdots (n-i+1)$. Lido de trás para a frente, esse é um produto de i inteiros consecutivos começando em $n-i+1$, ou seja, $N_i = P(i,\, n-i+1)$; por exemplo, $10 \cdot 9 \cdot 8 = 8 \cdot 9 \cdot 10 = P(3, 8)$. Como $i \leq k \leq n$, o início $n-i+1$ é pelo menos 1, e o lema garante que $i!$ divide N_i.

Isso explica a Tabela 1. Antes da i-ésima divisão, o acumulado já foi dividido por $1, 2, \ldots, i-1$, ou seja, por $(i-1)!$. Então

$$\underbrace{\frac{N_i}{(i-1)!}}_{\text{antes da divisão}} \div i \;=\; \frac{N_i}{i!} \;=\; \binom{n}{i},$$

que é inteiro pelo lema. No passo 3 da tabela, por exemplo, $720/2! = 360$ e $360/3 = 120 = 720/3!$. Por isso o denominador não precisa enviar $i!$: basta enviar o próximo fator i, e as divisões sucessivas por $1, 2, \ldots, i$ têm o mesmo efeito que dividir por $i!$. Em $\binom{30}{15}$, o maior valor acumulado fica em torno de $2{,}3 \times 10^9$, contra $2{,}0 \times 10^{20}$ do numerador completo.

2.3 Processos, Dados Compartilhados e Condições de Corrida

O programa tem três *threads*:

- **Denominador** (T_d): gera, em ordem, os fatores $1, 2, \ldots, k$ e os escreve num *buffer* compartilhado;

- **Numerador** (T_n): para $i = 1, \ldots, k$, multiplica o seu acumulado por $n - i + 1$, lê o próximo fator do *buffer* e divide o acumulado por ele;
- **Principal**: lê n e k, cria T_d e T_n, espera as duas terminarem (`pthread_join`) e imprime o resultado.

O acumulado pertence só a T_n (a principal só o lê depois do `pthread_join`), então não há disputa por ele. O que é compartilhado é o *buffer*: as posições de memória onde T_d escreve os fatores e de onde T_n os lê, junto com as variáveis que indicam quantos fatores estão disponíveis. Sem sincronização, ele está sujeito a três condições de corrida:

1. **Leitura antes da escrita:** T_n lê uma posição em que T_d ainda não escreveu e obtém um valor antigo ou o valor inicial (se for 0, divide por zero).
2. **Sobrescrita:** T_d escreve um fator por cima de outro que T_n ainda não leu, e esse divisor se perde.
3. **Atualização perdida:** se as duas *threads* alteram a mesma variável de controle (por exemplo, `qtd++` em T_d e `qtd--` em T_n), cada operação é feita em três passos (ler, alterar, escrever), e uma alteração pode apagar a outra.

A Tabela 2 mostra uma execução de $\binom{5}{2} = 10$ sem sincronização, com um *buffer* de uma posição, em que as corridas 1 e 2 acontecem e o programa imprime 4 sem nenhum erro aparente.

Tabela 2: Execução sem sincronização de $\binom{5}{2}$, passo a passo.

Passo	T_d	*buffer*	T_n	Acumulado	Problema
1	–	–	multiplica por 5	5	
2	escreve 1	1	–	5	
3	escreve 2	2	–	5	o 1 se perdeu (corrida 2)
4	–	2	lê 2 e divide: 5/2	2	divisão truncada
5	–	2	multiplica por 4	8	
6	–	2	lê 2 de novo: 8/2	**4**	valor antigo (corrida 1)

Uma solução correta deve garantir:

- **entrega exata:** cada fator é lido por T_n exatamente uma vez, na ordem em que foi escrito;
- **exclusão mútua** no acesso às variáveis do *buffer*;
- **término sem *deadlock*:** as duas *threads* terminam após k iterações.

Com as duas primeiras, T_n repete exatamente a Tabela 1 e, pelo Lema 1, obtém $\binom{n}{k}$. Também é desejável que as esperas sejam por bloqueio, e não por espera ocupada.

3 Descrição do Algoritmo Desenvolvido para Solução do Problema Proposto

3.1 Visão Geral

3.2 Mecanismos de Sincronização Utilizados

3.3 Pseudocódigo

3.4 Corretude

3.5 Implementação

3.6 Testes e Resultados

4 Conclusão

5 Referências

BEN-ARI, Mordechai. **Principles of Concurrent and Distributed Programming**. 2. ed. Harlow: Addison-Wesley, 2006. ISBN 978-0-321-31283-9.